

Spring Security Workflow — Complete Notes

Filter Chain → AuthenticationManager → AuthenticationProvider → UserDetailsService → Database, explained step-by-step with a working login example

1. The Big Picture (In Simple Words)

Think of Spring Security like the security desk of a large office building. Every visitor (HTTP request) has to pass through the front desk (Filter Chain) before reaching any office (your controller/API). The front desk doesn't decide by itself whether to let someone in — it calls a manager (AuthenticationManager), the manager checks with the right department (AuthenticationProvider), and that department looks up the visitor's ID card details in a records office (UserDetailsService), which pulls the actual record from a filing cabinet (Database). Once verified, the visitor gets a badge (SecurityContext) that they carry for the rest of their visit.

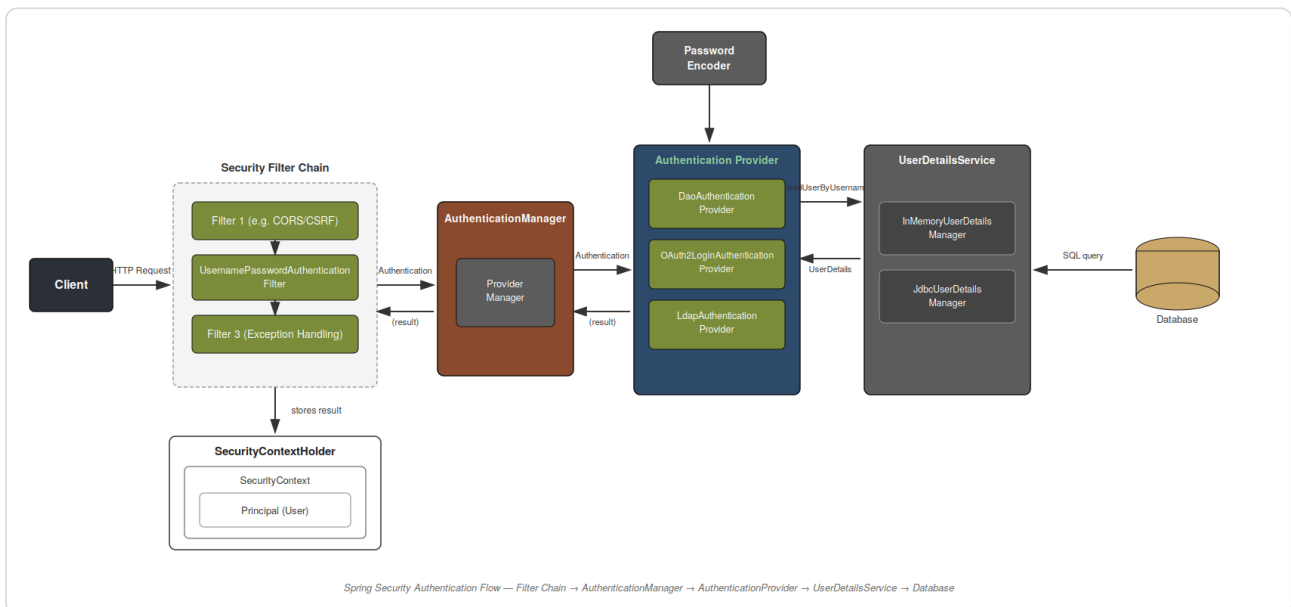


Fig 1. Clean recreation of the Spring Security authentication workflow

2. Step-by-Step Explanation of the Flow

- 1 Client sends a request.** A user tries to log in or access a protected resource, e.g. `POST /login` with a username and password, or a request carrying a JWT/session cookie.
- 2 Request enters the Security Filter Chain.** Spring Security is built as a chain of servlet filters. Every incoming request passes through this chain *before* it reaches your controller. Filters run one after another in a fixed order.
- 3 UsernamePasswordAuthenticationFilter (or equivalent) intercepts the credentials.** For a normal login form, this filter reads the username and password from the request and packages them into an `Authentication` object

(specifically `UsernamePasswordAuthenticationToken`), which at this point is *unauthenticated* — it just holds the raw credentials.

4 The filter hands off the Authentication object to the AuthenticationManager. The filter itself never checks passwords — it delegates that job by calling `authenticationManager.authenticate(authRequest)`.

5 AuthenticationManager (ProviderManager) picks the right AuthenticationProvider. Spring Security may support several authentication mechanisms at once (form login, OAuth2, LDAP). The `ProviderManager` is the default implementation of `AuthenticationManager`; it loops through a list of registered `AuthenticationProvider`s and asks each one: "can you handle this type of Authentication?"

6 The matching AuthenticationProvider does the real verification. For a normal DB-backed login, that's `DaoAuthenticationProvider`. It does two things:

- Calls `UserDetailsService.loadUserByUsername(username)` to fetch the stored user record.
- Uses the `PasswordEncoder` to compare the submitted password against the stored (hashed) password.

Other providers exist for other flows — `OAuth2LoginAuthenticationProvider` for social/OAuth2 login, `LdapAuthenticationProvider` for LDAP directories.

7 UserDetailsService fetches the user from storage. This could be an in-memory list (`InMemoryUserDetailsManager`, mostly for testing/demos) or a real database via `JdbcUserDetailsManager` or, most commonly in real apps, a custom implementation backed by JPA/Hibernate that queries the database.

8 Password check happens with PasswordEncoder. Passwords are never stored in plain text. The encoder (e.g. `BCryptPasswordEncoder`) hashes the incoming raw password and compares it against the stored hash using `matches()`.

9 Result flows back up. If the credentials are valid, the provider returns a fully populated, *authenticated* `Authentication` object (now containing the user's authorities/roles). This flows back: Provider → AuthenticationManager → Filter. If invalid, an `AuthenticationException` is thrown instead and the request is rejected (401).

10 The authenticated Authentication object is stored in the SecurityContext. The filter places it inside `SecurityContextHolder` → `SecurityContext` → `Authentication (Principal)`. From this point on, for the rest of the request (and for a session-based app, for the whole session), Spring Security knows exactly who the user is and what roles/authorities they have.

11 Remaining filters run, then the request reaches your controller. Later filters use the `SecurityContext` to enforce authorization — checking `@PreAuthorize`, role-based rules, CSRF, etc. — before your business logic ever executes.

Key idea to remember: Filters never validate credentials themselves. They only collect the request and delegate to the `AuthenticationManager`, which delegates to the right `AuthenticationProvider`, which delegates to `UserDetailsService` for data and

PasswordEncoder for hash comparison. Each component has exactly one job — this is the Single Responsibility Principle applied to security.

3. Who Talks to Whom (Quick Reference)

Component	Responsibility	Talks To
Filter Chain	Intercepts every HTTP request	AuthenticationManager
AuthenticationManager (ProviderManager)	Delegates authentication to the correct provider	AuthenticationProvider(s)
AuthenticationProvider	Performs actual credential verification	UserDetailsService, PasswordEncoder
UserDetailsService	Loads user record by username	Database / In-memory store
PasswordEncoder	Hashes & compares passwords	AuthenticationProvider
SecurityContextHolder	Stores the authenticated user for the current thread/session	Rest of the application

4. Full Working Example: User Login Flow End-to-End

Below is a complete, minimal Spring Boot + Spring Security setup that follows the *exact same flow* explained above — from the client hitting `/login`, through the filter chain, AuthenticationManager, AuthenticationProvider, UserDetailsService, down to the database — with every important annotation and line commented so you know exactly what is happening and why.

4.1 Entity — the row that comes from the Database

```
@Entity // Marks this class as a JPA entity mapped to a DB table
@Table(name = "users") // Maps to the "users" table
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    // IMPORTANT: this column stores the BCrypt HASH, never the raw password
    private String password;

    // e.g. "ROLE_USER", "ROLE_ADMIN" – comma separated or a separate table
    private String role;

    // standard getters and setters ...
}
```

4.2 UserDetailsService — fetches the record from the Database (Step 7 in the flow)

```
@Service // Registers this class as a Spring-managed bean
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository; // Spring Data JPA repo -> talks to the DB

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {

        // 1. This method is called AUTOMATICALLY by DaoAuthenticationProvider
        //     (Step 6 in the flow) – you never call it yourself.
        User user = userRepository.findByUsername(username)
            .orElseThrow(() ->
                new UsernameNotFoundException("User not found: " + username));

        // 2. Convert your DB entity into Spring Security's own UserDetails object.
        //     org.springframework.security.core.userdetails.User is a ready-made
        //     implementation of the UserDetails interface.
        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(), // the BCrypt hash from the DB
            List.of(new SimpleGrantedAuthority(user.getRole()))
        );
    }
}
```

4.3 PasswordEncoder Bean — used for hashing & comparison (Step 8)

```
@Configuration
public class SecurityConfig {

    @Bean                                // Exposes BCryptPasswordEncoder as a Spring bean
    public PasswordEncoder passwordEncoder() {
        // BCrypt automatically salts and hashes; "strength" 10 is the default work factor
        return new BCryptPasswordEncoder();
    }

    // -----
    // AuthenticationProvider – Step 6: wires UserDetailsService +
    // PasswordEncoder together so it can verify credentials.
    // -----
    @Bean
    public DaoAuthenticationProvider authenticationProvider(
        CustomUserDetailsService userDetailsService,
        PasswordEncoder passwordEncoder) {

        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(userDetailsService); // where to fetch the user
        provider.setPasswordEncoder(passwordEncoder);       // how to check the password
        return provider;
    }

    // -----
    // AuthenticationManager – Step 5: the entry point filters call.
    // Spring builds a ProviderManager under the hood using the
    // AuthenticationConfiguration, which already knows about the
    // DaoAuthenticationProvider bean defined above.
    // -----
    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    // -----
    // SecurityFilterChain – Step 2 & 3: configures the filter chain
    // itself: which endpoints are public, which need auth, and how
    // login is handled.
    // -----
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // disabled here for a stateless API
demo
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login", "/register").permitAll() // public endpoints
                .anyRequest().authenticated() // everything else needs login
            )
            .formLogin(form -> form.disable()) // we'll use our own /login below
            .sessionManagement(sm -> sm
                .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED));

        return http.build();
    }
}
```

4.4 Login Controller — where the Client's request lands first (Step 1 & 4)

```
@RestController // Handles incoming HTTP requests, returns JSON
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager; // Step 5 entry point

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest request) {

        // 1. Package the raw credentials into an UNauthenticated Authentication
        //    object. This mirrors what UsernamePasswordAuthenticationFilter
        //    normally does automatically for a form login.
        UsernamePasswordAuthenticationToken authRequest =
            new UsernamePasswordAuthenticationToken(
                request.getUsername(),
                request.getPassword());

        // 2. Hand it to the AuthenticationManager – Step 4 & 5.
        //    Internally this calls ProviderManager -> DaoAuthenticationProvider
        //    -> CustomUserDetailsService.loadUserByUsername() -> PasswordEncoder.matches()
        //    If credentials are wrong, this line THROWS BadCredentialsException.
        Authentication authResult = authenticationManager.authenticate(authRequest);

        // 3. Store the now-AUTHENTICATED Authentication object in the context –
        //    Step 10. Everything downstream (this request/session) now knows
        //    who the logged-in user is.
        SecurityContextHolder.getContext().setAuthentication(authResult);

        // 4. (Optional but common) generate a JWT here so the client can send it
        //    on every future request instead of relying on a session cookie.
        String token = jwtService.generateToken(authResult);

        return ResponseEntity.ok(Map.of("token", token));
    }
}
```

Where things fail: If the username doesn't exist, `loadUserByUsername()` throws `UsernameNotFoundException`. If it exists but the password hash doesn't match, `DaoAuthenticationProvider` throws `BadCredentialsException`. Both are subclasses of `AuthenticationException`, which Spring Security's exception-handling filter (Filter 3 in the diagram) catches and converts into an HTTP **401 Unauthorized** response — your controller code never has to handle this manually.

5. Tracing One Real Login Request Through the Diagram

Putting it all together — here is exactly what happens when a user logs in with `username=suraj` and `password=mypassword123`:

1. **Client → Filter Chain:** Browser sends `POST /login` with the JSON body. It enters the Security Filter Chain.
2. **Filter → Controller → AuthenticationManager:** Because we used a custom controller (section 4.4), the request reaches `AuthController.login()`, which builds an unauthenticated token and calls `authenticationManager.authenticate(...)`.
3. **AuthenticationManager → AuthenticationProvider:** `ProviderManager` finds that `DaoAuthenticationProvider` supports `UsernamePasswordAuthenticationToken` and delegates to it.

4. **AuthenticationProvider** → **UserDetailsService** → **Database**: `DaoAuthenticationProvider` calls `loadUserByUsername("suraj")`, which runs a query like `SELECT * FROM users WHERE username = 'suraj'` and returns the row, including the stored BCrypt hash.
5. **AuthenticationProvider** → **PasswordEncoder**: `passwordEncoder.matches("mypassword123", storedHash)` hashes the raw password and compares it against the stored hash.
6. **Result flows back**: Match found → an authenticated `Authentication` object (with role `ROLE_USER`) is returned all the way back up to the controller.
7. **SecurityContextHolder**: The controller stores this authenticated object via `SecurityContextHolder.getContext().setAuthentication(authResult)`.
8. **Response**: A JWT (or session) is generated and returned to the client, who will attach it to every future request so the same identity can be re-established without logging in again.

In one line: The Client never talks to the Database directly — every step is a hand-off between single-responsibility components, and the diagram in Section 2 is literally the call stack of one `/login` request.